🧠 DAY 0 — WEBSITE DATA FLOW MASTER GUIDE

(Dynamic X-Ray — No Code Day)

> **Goal:** Before writing a single line of code, decide exactly how the website delivers data.

If you finish this document, you must be able to fill:

```
Site:
Type:
Rendering Style:
Data Endpoint:
Method:
Pagination:
Security:
Replayable?:
Best Strategy:
```

If you can't — Day 0 is not complete.

🔥 CORE MENTAL MODEL (NON-NEGOTIABLE)

Web scraping ≠ copying pages. Web scraping = **reverse-engineering a data pipeline**.

You are not scraping.

You are answering:

> "How does this website breathe?"

If you can narrate the data journey in 2–3 sentences, you have won.

🧭 STEP 0 — DEFINE THE REAL GOAL

Before opening DevTools:

Ask yourself:

- What exact data do I need?
- Listing page data?
- Detail page data?
- Search results?
- Authenticated data?
- One page or 10,000 pages?

Diagnosis without goal = wandering.

🧱 STEP 1 — SERVER VS BROWSER (Rendering Style)

There are only two rendering models.

🟢 A) Server-Side Rendering (SSR)

Server sends full HTML with real data inside.

Diagnosis:

- View Page Source → you see real data
- Disable JavaScript → page still works

Conclusion:

```
Rendering Style: SSR
Type: Static HTML
Best Tool: requests + BeautifulSoup
```

🔵 B) Client-Side Rendering (CSR)

Server sends empty skeleton. JavaScript fetches data and builds DOM.

Diagnosis:

- View Page Source → mostly empty
- Elements tab → full data
- Network tab → JSON calls

Conclusion:

```
Rendering Style: CSR
Data comes via API
Tool depends on replayability
```

🔬 STEP 2 — DEVTOOLS X-RAY

Open:

```
DevTools → Network → Filter: XHR / Fetch
```

Reload page.

Observe:

- Request URL (endpoint)
- Method (GET / POST)
- Status code (200 / 403 / 429)
- Content-Type
- Response body

You are watching data travel.

📦 STEP 3 — IDENTIFY DATA DELIVERY MECHANISM

There are only 5 real patterns.

1️⃣ Pure HTML (Static)

Signs:

- Network mostly “document”
- No JSON calls

- Data in page source
- Pagination like:
    - `/page/2`
    - `?page=2`

```
Type: Static HTML
Method: GET
Replayable: Yes
Best Strategy: requests
```

---

2 REST API (JSON over HTTP)

Signs:

- GET request
- `/api/products?page=2`
- Response = JSON
- Content-Type: application/json

Pagination styles:

- page=2
- offset=20
- limit=50

```
Type: REST API
Method: GET
Replayable: Usually yes
Best Strategy: requests or aiohttp
```

---

3 GraphQL

Signs:

- POST request
- Endpoint `/graphql` or `/`
- Request body contains `"query"`
- Response wrapped in `"data": {}`

Pagination:

- cursor
- after
- first
- edges

```
Type: GraphQL
Method: POST
Replayable: Often yes
Best Strategy: requests (POST JSON)
```

---

4 Token / Cookie Protected API

Signs:

- JSON API exists
- But requires:
    - cookies
    - CSRF token
    - Authorization header

Without headers → 403 or empty response.

```
Type: Hybrid
Replayable: Partially
Best Strategy: Browser capture → replay API
```

---

5 Fully Browser-Dependent (Anti-bot heavy)

Signs:

- Encrypted payload
- Dynamic signature
- Fingerprinting headers
- Replay fails even with copied headers

```
```

```
Type: Browser-only
Replayable: No
Best Strategy: Playwright only
```

STEP 4 — PAGINATION ANALYSIS

Pagination determines scalability.

Look for:

- `?page=2`
- `offset=20`
- `start=50`
- `"cursor": "abc123"`
- Infinite scroll (same endpoint repeated)

Types:

1. Page-based
2. Offset-based
3. Cursor-based
4. Infinite scroll
5. No pagination

Important:

Cursor ≠ page number. Cursor = pointer token to next batch.

STEP 5 — SECURITY LAYER

Click request → Headers tab.

Look for:

- Cookie
- Authorization
- x-csrf-token
- x-api-key
- Custom headers
- Rate limit responses (429)

- CAPTCHA
- Cloudflare

Security ≠ rendering type.

STEP 6 — REPLAY TEST

Copy request → Copy as cURL.

Test outside browser.

If works:

Replayable → direct API scraping.

If fails:

Requires session.

If still fails:

Browser-only.

---

5 LAYERS OF WEBSITE CLASSIFICATION

Every site must be classified across these independent layers:

Layer	Question
Rendering	Where HTML built?
Data Mechanism	How data delivered?
Pagination	How next batch fetched?
Security	What protects endpoint?
Replayability	Can it run outside browser?

Keep them separate.

🧠 PROFESSIONAL CLASSIFICATION FORMAT

Never say:

❌ "It is REST."

Instead say:

```
Site:
Rendering: CSR
Data Mechanism: REST API
Data Endpoint: /api/search
Method: GET
Pagination: offset-based
Security: cookies + csrf
Replayable: partially
Best Strategy: hybrid (browser + aiohttp)
```

That is real diagnosis.

🧠 MASTER RULE

There are only 3 real scraping situations:

1. Data already in HTML → Easy
2. Data available via API → Better
3. Data protected via session → Hybrid

Everything else is variation.

🎯 COMPLETE DIAGNOSIS FLOW

Follow this every time:

1. View Page Source
2. Check if data present
3. Open Network → XHR
4. Identify endpoint

5. Identify method
6. Inspect headers
7. Check pagination
8. Replay test
9. Classify before coding

No Python until classification is done.

🧠 STATUS CODES YOU MUST READ

Code	Meaning
200	OK
301	Redirect
403	Forbidden
429	Too Many Requests

🧠 DAY 0 DONE WHEN

You can:

- Diagnose site type in ≤ 10 minutes
- Explain data flow in one sentence
- Choose correct tool before coding
- Fill this without confusion:

```
Site:
Type:
Rendering Style:
Data Endpoint:
Method:
Pagination:
Security:
Replayable?:
Best Strategy:
```

🎯 FINAL DAY 0 CHECKLIST

☑ Open DevTools ☑ Identify HTML vs API ☑ Identify endpoint ☑ Identify method ☑ Identify pagination ☑ Identify security ☑ Test replay ☑ Choose minimal tool

---